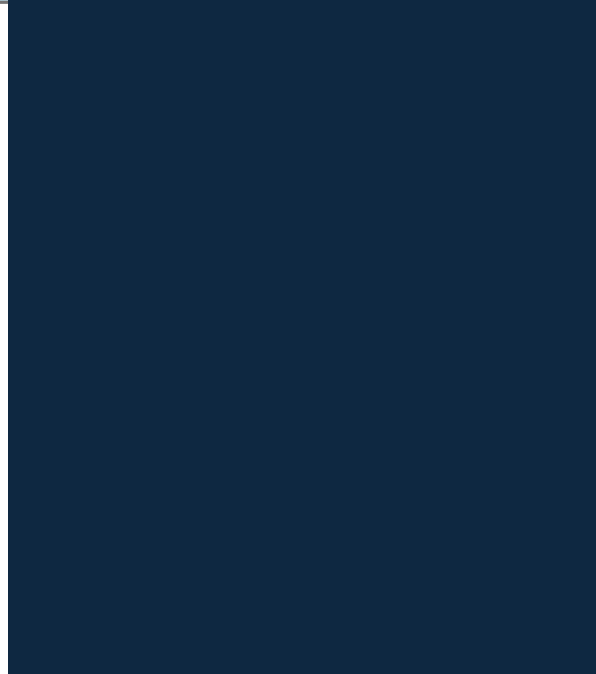




Macroinvertebrate EDA Program

Implementation Summary

Ajugo Maxwell Vuni
Jai Butler



Project title and group members

Macroinvertebrate EDA Program by Ajugo Maxwell Vuni & Jai Butler

Brief project goal

Our goal was to achieve a fully functional and efficient program that takes the Kaggle macroinvertebrate dataset, performs exploratory data analysis (EDA), and provides the information the user in an accessible and layperson-friendly manner, using tkinter. We wanted the EDA portion of the program to provide basic statistical data about the images, as well as graphical data.

Summary of the system design

The system was designed as a modular Python application, with each responsibility separated into its own class and module. Classes communicate through a shared workflow service. The indexer produces a DataFrame, the EDA service consumes it, and the workflow service coordinates both without the GUI needing to know any implementation details. A shared config.py centralises all directory paths and settings to avoid hard-coded values throughout the scripts.

Class and module overview

This program uses four custom classes, integrated into a consistent program. The **DatasetIndexer** class reads the raw image data and formats it into a pandas DataFrame for use by the other classes. The **EDAService** is an exploratory data analysis class that creates and outputs visual data using the indexed dataset. The **WorkflowService** class integrates the output of both earlier classes and cleans up the outputs before generating them. It also uses the DatasetIndexer to create the DataFrame upon object initialisation. The **GUI_App** class hosts the code for the tkinter GUI. It defines and manages the buttons and output field.

Tools and libraries used

The program was developed in Python using VS Code. The libraries we used to implement the functionality of the program were:

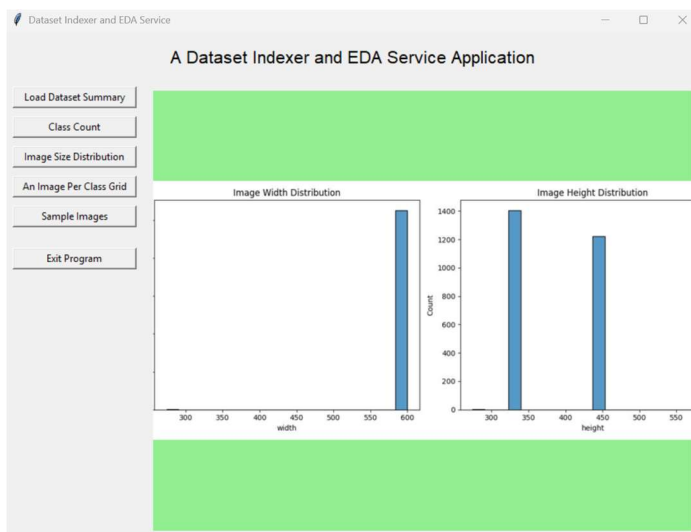
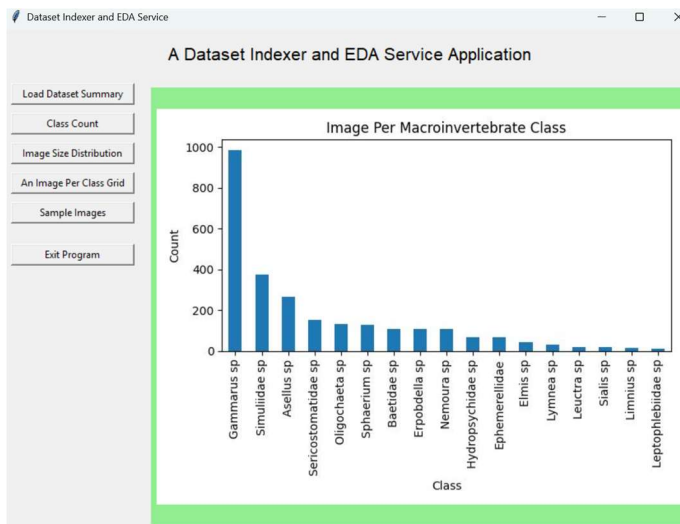
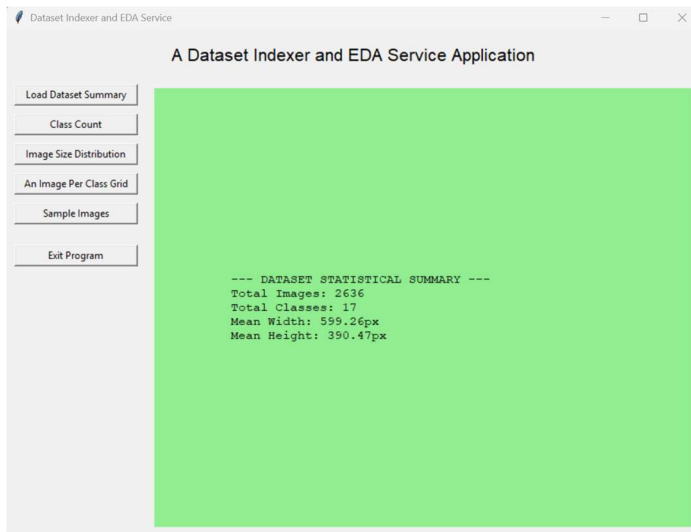
- sys
- pathlib
- pandas
- opencv-python
- matplotlib
- seaborn
- tkinter
- Pillow

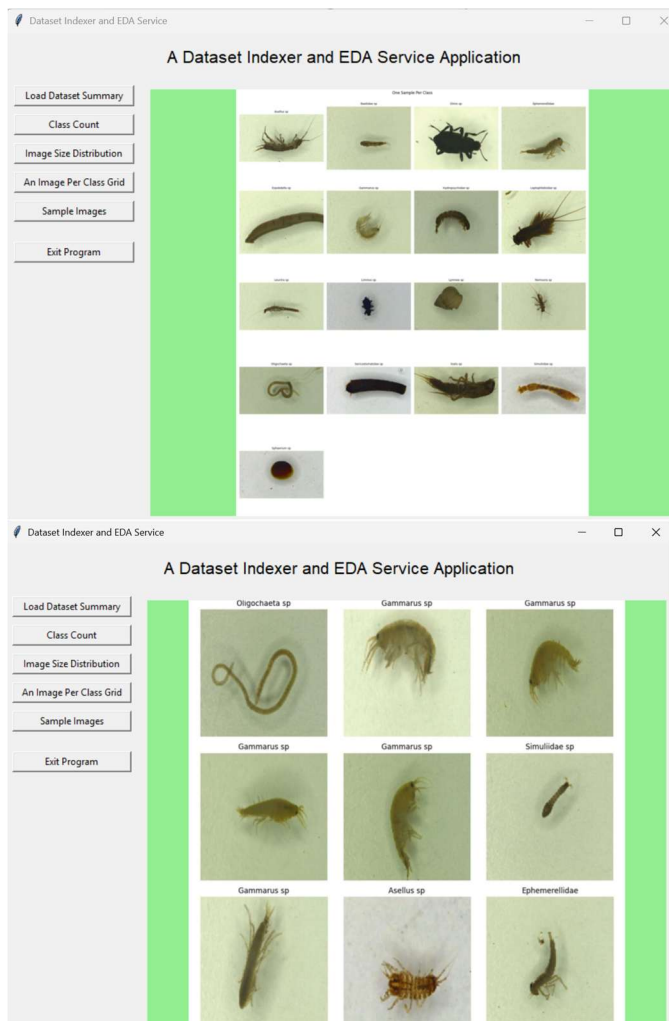
Key features implemented

The program provides a simple easy-to-use GUI for the user to interact with. Within this GUI, the user can do the following:

- **Load the dataset summary:** prints a simple summary to the GUI output field with the total number of images, the total number of classes, and the mean width and height of the images.
- **Generate a class count graph:** generates, saves (to the EDA output directory), and displays a bar chart showing the distribution of classes (i.e. how many images per macroinvertebrate class).
- **Generate an image size distribution chart:** generates, saves, and displays a histogram showing the distribution of image widths & heights.
- **Generate an image per class grid:** generates, saves, and displays a grid of 17 images, with one selected at random from each class. Each time this option is selected, it generates a new grid with a new random image from each class.
- **Generate a sample image grid:** generates, saves, and displays a grid of 9 images, with each image selected at random from the dataset. Each time this option is selected, it generates a new grid with new random images.
- **Exit:** terminates the program.

Screenshots or example outputs





Testing summary

Each module was tested independently during development, with its required dependencies, to ensure it produced the expected output before moving on. Once all classes were combined into a consistent workflow, each Tkinter button was tested to confirm it produced the correct result (e.g. text summaries displaying as formatted text and charts rendering as images in the result area).

One issue encountered during testing was that the initial Text widget used for the output area cropped images incorrectly. After investigation, this was resolved by replacing it with a Label widget and using Tkinter's image configuration to display chart outputs. Chart generation and file saving to the correct output directory worked as expected with no issues.

It is important to note that the program must be launched from an IDE that supports opening a project folder, such as VS Code, rather than running main.py in isolation. This

is necessary because the program relies on relative file paths that are resolved from the project root. Launching the file directly without the correct working directory will cause path resolution errors.

Acknowledgement of reused or adapted code

We reused and adapted code provided in the 'Full Guidance and Coding Examples' document provided for the purposes of this assignment, as well as some pieces of code used for assignment 2.

Short summary of how work was divided among group members

Ajugo Maxwell was responsible for the design and implementation of the Dataset Indexer & the GUI portions of the program. Jai designed and implemented the EDA Service as well as the overall program integration. Collaboration was achieved through progress checkup chats via discord and reviewing of each other's code to ensure functionality & usability when integrated with other code.

Additionally, Ajugo Maxwell took the responsibility of preparing the presentation, whilst Jai developed this implementation summary.